

AMBIENCE

Text-to-Ambience

Un prompt en lenguaje natural → una definición de ambiente (JSON) vía un pipeline RAG + LLM, persistida y visualizada.

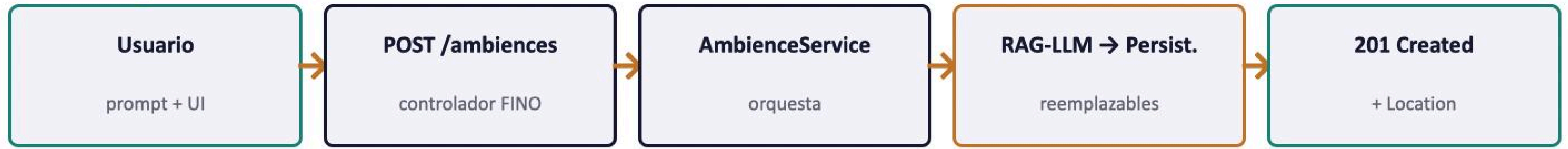
FastAPI · Python 3.12

Vanilla HTML/CSS/JS

Docker

.claude/ = producto

Qué se construyó y cómo se organiza



Responsabilidad por módulo

- › **api/** controladores finos: validan, llaman 1 método, responden.
- › **services/** orquestan casos de uso; dependen de interfaces.
- › **rag/** caja RAG-LLM reemplazable (PromptBuilder · Retriever · LLMProvider).
- › **persistence/** caja de almacenamiento reemplazable (AmbienceStore).
- › **models/** esquemas Pydantic = el contrato.
- › **core/ · ratelimit/ · usage/** config + DI, rate-limit y cuota mensual (SQLite).

Features de Claude Code (1–4)

1 CLAUDE.md

Memoria de proyecto: folder map + convenciones, cargada en cada sesión.

Anidado en app/rag/: se auto-carga solo en la caja RAG.

2 rules/ path-scoped

api.md aplica solo al HTTP (app/api).
+ code-style · testing · security · git globales.

3 settings.json — permissions

allow: repo, git read-only, pytest/ruff/mypy.

deny .env/secrets · **ask** commit, pip, docker, rm.

4 Skill + slash command

add-endpoint: modelos → controlador → servicio → tests → OpenAPI.

Invocada por **/add-endpoint**.

Features de Claude Code (5–7)

5

Sub-agent: rag-researcher

Investiga opciones RAG/LLM (proveedores, retrieval, costo/latencia).

Recomendaciones, no código; tools acotadas.

6

Plan mode (vivencia)

brainstorm → plan → worktree → TDD → review.

Specs + planes en docs/superpowers/.

7

El propio .claude/ es un entregable

El .claude/ en acción

CLAUDE.md + rules

Consistencia automática: controladores finos e interfaces, sin repetirlo.

permissions

Seguridad por defecto: .env bloqueado; mutaciones piden confirmación.

skill / command

Endpoints completos (modelos + controlador + servicio + tests + OpenAPI) en un paso.

sub-agent

Decisiones de la caja RAG informadas, sin contaminar el contexto principal.

plan mode

Features por slices con spec + plan antes de tocar código (TDD).

OpenAPI 3.1 · Swagger · /ambiences

Ambience API OAS 3.1

POST

/ambiences

Request body

```
{ prompt: 3-2000, user_id }
```

Responses

201

Created · Ambience + Location

422

Validation Error

Schemas

AmbienceRequest

prompt string · 3-2000

user_id string · min 1

Ambience (response)

uuid · fingerprint · created_at

name · intension · publico

filters[] → Filter → Genre/Range

openapi.yaml se regenera con **/add-endpoint** · Swagger UI en **/docs**

Demo

Del prompt al ambiente — el pipeline completo, en marcha.

1

Prompt “rainy afternoon jazz...” → POST /ambiences

2

Pipeline RAG-LLM genera y valida el JSON

3

Respuesta 201 Created + Location, persistido

4

Frontend visualización (EN/ES auto)